La gestione della memoria centrale (RAM)

La gestione della memoria centrale è una funzione fondamentale di qualsiasi Sistema Operativo e consiste nell'organizzare l'uso della memoria RAM tra i vari processi in esecuzione, in modo efficiente e sicuro. La memoria RAM viene usata per caricare programmi, dati e per fornire lo spazio necessario all'esecuzione dei processi.

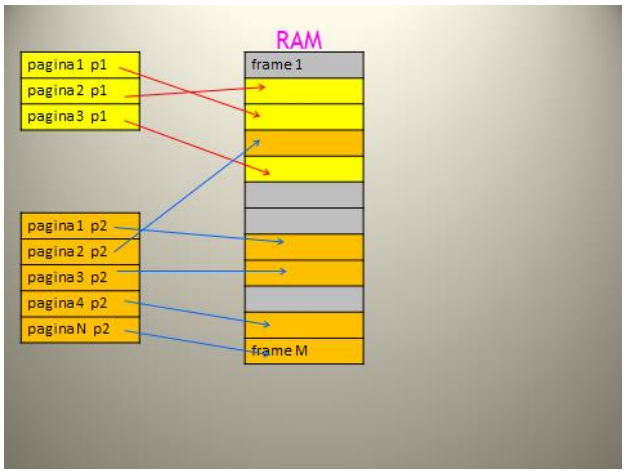
La RAM è, ovviamente, una risorsa limitata; per questo il S.O. attua diverse strategie per sfruttarla al meglio.

Come sappiamo, i moderni Sistemi Operativi lavorano sfruttando la multiprogrammazione, che ricordiamo prevede la presenza di più processi, oltre al S.O., in memoria RAM.

Quindi, quando il codice di un programma viene compilato per generare il file eseguibile (e quindi eseguire un processo), il compilatore non può sapere a priori in quale porzione di memoria RAM verrò caricato il codice macchina.

Quello che accade oggi è che la RAM viene gestita suddividendola in blocchi di dimensione fissa, detti **frame**, di solito di 4 KB; anche lo spazio occupato dai programmi viene suddiviso in porzioni della stessa dimensione dei frame, chiamati **pagine**.

Ogni programma lavora in uno **spazio di indirizzamento logico**, dove gli indirizzi vanno da 0 fino all'indirizzo dell'ultimo byte dell'ultima pagina, e vede il proprio codice come se fosse contiguo. In realtà, ogni pagina verrà poi caricata in uno **spazio di indirizzamento fisico** della RAM, occupando un qualsiasi frame libero; pertanto, non è detto che le pagine siano memorizzate in frame contigui tra di loro, e nemmeno che rispettino il loro ordine.



Inoltre, pagine usate molto raramente potrebbero essere mantenute su disco e caricate in RAM solamente quando davvero necessario, in modo da ottimizzare l'utilizzo della memoria.

Possiamo capire, quindi, che gli **indirizzi logici** non corrispondano agli effettivi **indirizzi fisici** della memoria. Tuttavia, tutto questo deve essere

trasparente sia per i programmatori che per i processi stessi: la memoria del nostro programma parte dall'indirizzo 0 (o un altro valore fisso), si sviluppa in modo continuativo, termina con l'indirizzo dell'ultimo byte dell'ultima pagina ed è sempre caricato in memoria, anche se in realtà niente di tutto ciò è necessariamente vero.

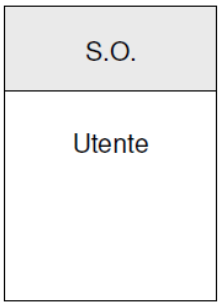
Naturalmente è il Sistema Operativo a gestire tutto questo: viene supportato da un dispositivo hardware, l'**MMU** (Memory Management Unit), che si occupa di tradurre gli indirizzi logici in indirizzi fisici.

Analizziamo brevemente le diverse tecniche di gestione della memoria centrale che si sono susseguite nel corso degli anni.

Partizione unica e contigua

La memoria RAM è suddivisa in sole due parti:

- una parte riservata al Sistema Operativo;
- una parte utilizzabile da tutti gli altri programmi (o meglio dire processi).



Non si ha multiprogrammazione, pertanto un solo processo alla volta può essere presente in memoria RAM; dal momento che quest'ultima è divisa in due parti, l'indirizzo iniziale per il processo utente caricato in RAM sarà sempre lo stesso.

Non ci si pone il problema di dover tradurre gli indirizzi logici in indirizzi fisici, in quanto questi coincidono.

Possiamo facilmente intuire i limiti di questo approccio (seppur molto semplice). Innanzitutto, non possiamo avere più di un processo per volta in memoria RAM, cosa che in genere porta ad uno spreco di memoria; infatti, anche se il processo non occupa tutto lo spazio di memoria disponibile, la parte restante rimane non utilizzata.

Inoltre, questo approccio di solito non prevede protezione della memoria, rischiando così che un processo utente vada ad accedere o peggio sovrascrivere spazi di memoria riservati al Sistema Operativo.

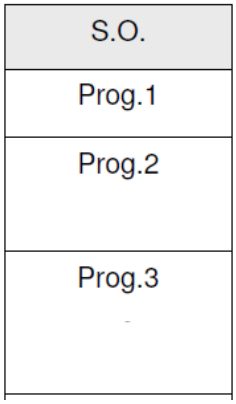
Questo approccio era utilizzato nei primi computer, quando i sistemi erano molto semplici e non si sentiva ancora l'esigenza della multiprogrammazione o della protezione tra processi (banalmente perché solo un processo per volta poteva essere in esecuzione).

Con l'evolversi delle tecnologie hardware e software, questo approccio è stato abbandonato a favore di altri più flessibili ed efficienti dal punto di vista della gestione della memoria.

Partizioni fisse

Il partizionamento fisso rappresenta una evoluzione rispetto alla singola partizione.

In questo caso, la memoria è suddivisa in un certo numero di partizioni, di dimensione fissa, ma non necessariamente uguale tra di loro. Questa suddivisione avviene all'avvio del sistema. Anche qui, una di queste partizioni è riservata al Sistema Operativo.



A differenza della partizione unica, qui viene introdotto un primo grado di multiprogrammazione, in quanto più processi alla volta possono essere presenti in memoria centrale contemporaneamente.

Il S.O. utilizzerà una tabella per gestire le partizioni: per ciascuna partizione memorizzerà l'indirizzo iniziale, l'indirizzo finale (o la lunghezza in byte della partizione) ed il PID del processo che attualmente la sta occupando.

N.B.: un solo processo può occupare una partizione.

La rilocazione degli indirizzi avviene solamente una volta, in fase di caricamento del processo in memoria centrale: il loader (che si occupa di caricare in memoria il codice di un processo) trasformerà gli indirizzi logici in indirizzi fisici andando ad aggiungere l'offset rispetto all'indirizzo iniziale della partizione nel quale il processo stesso viene caricato.

Per esempio, se l'indirizzo iniziale della partizione che ospiterà il processo è 1000, a tutti gli indirizzi logici presenti nel programma verrà sommato il valore 1000 per ottenere i relativi indirizzi fisici.

Questo approccio è sicuramente migliore rispetto al primo, ma presenta ancora dei difetti. Per prima cosa, il grado di multiprogrammazione, ovvero il numero di processi che possono essere contemporaneamente in memoria RAM, è fisso: N partizioni, N processi in memoria.

In secondo luogo, è soggetto ad un fenomeno, detto frammentazione interna: se il processo è più piccolo della partizione che lo ospita, lo spazio di memoria residuo rimane inutilizzato.

Infine, è impossibile il caricamento in memoria (e quindi l'esecuzione) di processi più grandi della partizione con dimensione maggiore, anche se la somma di tutte le partizioni lo consentirebbe.

Quest'ultimo problema è di difficile soluzione, in quanto non è possibile sapere in anticipo la dimensione dei processi che necessiteranno di essere caricati in memoria.

Per ovviare a questo inconveniente si potrebbe pensare di aumentare la dimensione delle partizioni; tuttavia, non solo questo ridurrebbe il grado di multiprogrammazione, consentendo di avere meno programmi in memoria (mantenendo invariata la quantità di memoria, aumentando la dimensione delle partizioni diminuisce il loro numero), ma potrebbe accentuare ancora di più il problema della frammentazione interna: infatti se la maggior parte dei programmi è molto piccola, lo spazio inutilizzato di ogni partizione risulta essere eccessivo, con uno spreco maggiore di memoria.

Il partizionamento fisso ha rappresentato un miglioramento rispetto alla gestione a partizione unica, ma ha presto mostrato i suoi limiti in termini di flessibilità (la dimensione delle partizioni è fissa e stabilita a priori) ed efficienza nell'utilizzo della memoria (frammentazione interna).

Partizioni variabili

Il partizionamento variabile rappresenta un ulteriore passo avanti nella gestione della memoria centrale, introducendo maggiore flessibilità rispetto al partizionamento fisso.

Inizialmente si hanno due sole partizioni: una riservata al S.O. e l'altra (tutto il resto della memoria) libera. Quando un processo deve essere caricato in memoria, viene creata dinamicamente una partizione con dimensione pari a quella del processo dividendo in due una delle partizioni libere presenti, che ovviamente deve essere sufficientemente grande.

Un processo può ancora essere caricato solamente in spazi di memoria contigui.

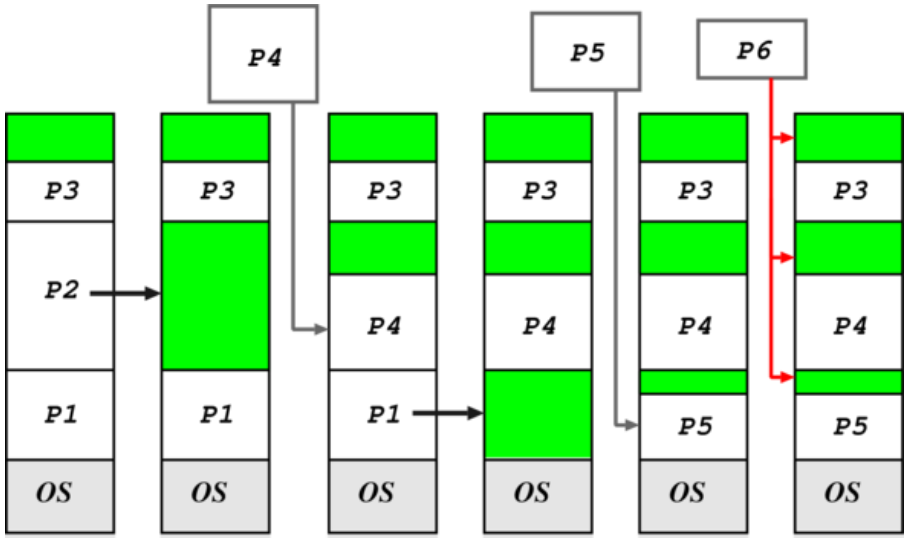
Quando un processo termina la sua esecuzione, la partizione da lui occupata viene liberata, accorpando, eventualmente, altre partizioni contigue che la precedono o seguono.

La memoria viene, quindi, partizionata di volta in volta in modo variabile, adattandosi alle reali necessità dei programmi.

Anche in questo caso il S.O. gestirà la memoria con una tabella molto simile a quella prevista nel partizionamento fisso, ma il numero di righe di questa

tabella sarà variabile, in quanto cambierà dinamicamente man mano che vengono aggiunte o rimosse delle partizioni. Questa gestione è decisamente più flessibile ed efficiente, in quanto non si ha un numero prestabilito di processi che possono essere caricati in memoria.

Inoltre, non si ha frammentazione interna: ogni partizione ha dimensione esattamente pari a quella del processo che ospita. Tuttavia, sorge un nuovo tipo di



problema, rappresentato dalla **frammentazione esterna**.

Questo fenomeno vede la creazione graduale di tante piccole partizioni libere, non contigue, che non sono in grado di contenere singolarmente un altro processo, anche se complessivamente magari sì.

Illustriamo l'esempio sopra raffigurato, che mostra come viene a crearsi frammentazione esterna. Inizialmente abbiamo tre processi (P1, P2 e P3); P2 termina la sua esecuzione, pertanto rilascia la partizione che lo ospitava.

Arriva un nuovo processo (P4), che trova due partizioni libere (evidenziate in verde), delle quali solamente la seconda, proprio quella liberata da P2, è grande abbastanza da contenerlo. Nel frattempo anche P1 termina l'esecuzione e rilascia la sua partizione, che è in grado di ospitare un nuovo processo (P5), che deve essere caricato in memoria.

A questo punto si presenta un ulteriore processo (P6), che trova tre partizioni libere; ognuna di esse è troppo piccola per poterlo ospitare, ma la loro dimensione, se sommata, è sufficiente. Tuttavia, P6 non può essere caricato in memoria, perché le partizioni non sono contigue tra di loro.

La scelta della partizione libera che ospiterà un certo processo può avvenire secondo tre diverse strategie:

1. **first fit**: cerca la prima partizione libera abbastanza grande per contenere il processo, scandendo la memoria a partire dall'inizio. È

rapida ed efficiente dal punto di vista del tempo di ricerca, ma può sviluppare frammentazione esterna nella parte iniziale della memoria, perché tende a riempire sempre i primi spazi disponibili;
2. **best fit**: il S.O. cerca tra tutte le partizioni libere quella la cui dimensione si avvicina di più a quella del processo da caricare in memoria (ovvero va a cercare la più piccola partizione libera in grado di ospitare il processo), in modo da sprecare meno spazio possibile. In teoria dovrebbe ridurre la frammentazione, ma in realtà tende a creare numerosi spazi inutilizzabili sparsi per la memoria, in quanto i resti lasciati risultano essere troppo piccoli per ospitare nuovi programmi. Inoltre, risulta essere più lenta della precedente, perché occorre analizzare tutte le partizioni libere;
3. **worst fit**: assegna il processo alla partizione più grande in grado di contenerlo, con l'idea che il blocco rimanente sia ancora sufficientemente grande da ospitare altri processi. Anche in questo caso l'operazione è più lenta, perché tutte le partizioni non occupate devono essere analizzate per trovare quella più grande. L'idea di questa strategia è quello di ritardare il più possibile la formazione di partizioni sempre più piccole.

In generale, la strategia worst fit si è rivelata la peggiore, perché, nel tentativo di preservare i blocchi piccoli, rovina la struttura dei blocchi più grandi, andando a "distruggere" quelle aree di memoria in grado di contenere processi di grandi dimensioni.

Tra first fit e best fit non esiste sempre una che prevarica l'altra, ma dipende dal contesto e dal tipo di sistema, anche se tipicamente si preferisce la prima per la sua estrema semplicità, il minor overhead introdotto (sono presenti meno calcoli e meno gestione da parte del S.O.) e la sua rapidità.

Per contrastare la frammentazione esterna, di solito i sistemi di questo tipo tendono ad utilizzare meccanismi di **garbage collection**; queste tecniche consentono di recuperare lo spazio delle partizioni libere, accorpandolo in un'unica partizione più grande, più facilmente utilizzabile. Per farlo, il Sistema Operativo sposta il codice dei processi attualmente presenti in memoria. Questo, però, porta a due problemi:

1. i meccanismi di garbage collection sono di solito molto onerosi in termini di tempo e risorse, provocando rallentamenti notevoli nel sistema;
2. ogni volta che il codice di un processo viene rilocato, tutti i riferimenti agli indirizzi utilizzati non sono più validi e devono essere ricalcolati, cosa che porta ad un ulteriore overhead.

Segmentazione semplice

La segmentazione semplice è una tecnica di gestione della memoria centrale che nasce con l'obiettivo di superare alcune limitazioni del partizionamento variabile (e delle tecniche precedenti); prima fra tutte il fatto che il programma debba essere logicamente lineare ed essere caricato in un'area contigua di memoria.

Con questa tecnica, infatti, ogni programma (processo) viene suddiviso in diversi **segmenti logici** (ad esempio uno per il codice, uno per i dati, uno per lo stack, …), ciascuno dei quali può essere caricato in una partizione diversa, senza la necessità che le partizioni siano contigue tra di loro.

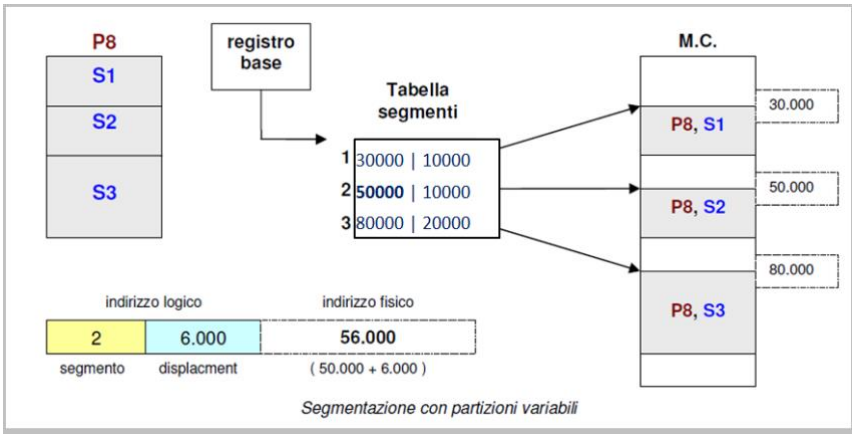
Questi segmenti riflettono meglio le strutture logiche del programma e non hanno necessariamente una dimensione fissa.

All'interno di ciascun segmento, gli indirizzi logici partono da 0; per andare ad individuare una istruzione o per fare riferimento ad un dato all'interno di un segmento occorre specificare il numero del segmento ed un **offset**, vale a dire un discostamento interno al segmento stesso.

La memoria centrale viene gestita sfruttando ancora le partizioni variabili; la principale differenza si trova nella tabella delle partizioni, nella qualche non viene più indicato solo il PID del processo, ma anche il numero del segmento di tale processo che è caricato in quella specifica partizione.

Il calcolo degli indirizzi fisici avviene dinamicamente, quando l'istruzione deve essere eseguita; la risoluzione degli indirizzi viene supportata da una **tabella dei segmenti,** prevista per ogni programma. Questa tabella ha tante righe quanti sono i segmenti in cui è stato diviso un programma e, per ogni segmento, viene indicato l'indirizzo (iniziale) della partizione nella quale è caricato e il numero di byte occupati.

Nel descrittore (PCB) di ogni processo sarà, inoltre, previsto un campo, detto **registro base**, che contiene un riferimento (l'indirizzo) alla sua tabella dei segmenti.

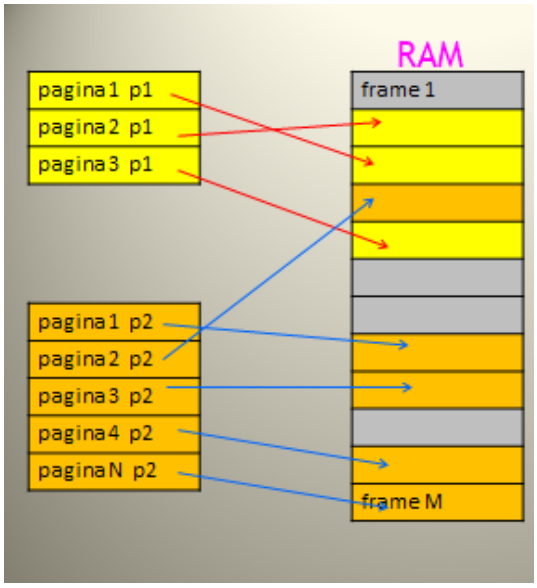


Segmentazione con partizioni variabili

Questa tecnica porta ad una migliore gestione della memoria, in quanto è più facile per un processo trovare delle partizioni in grado di contenere dei singoli segmenti anziché un blocco di codice intero; tuttavia, anche qui non manca il rischio di avere segmentazione esterna, in quanto un segmento deve comunque essere caricato in una singola partizione.

Paginazione semplice

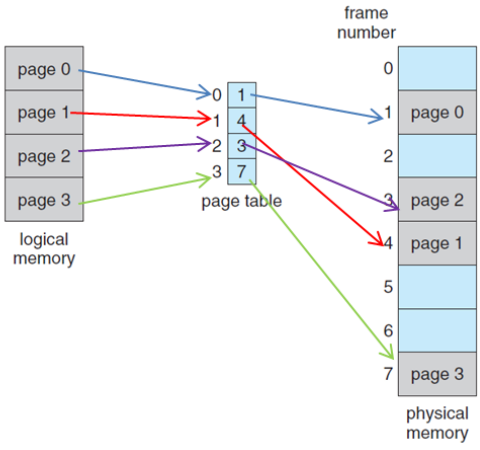
La paginazione semplice è una tecnica fondamentale nella gestione moderna della memoria centrale, introdotta per risolvere il problema della frammentazione esterna, che affligge le tecniche precedentemente utilizzate.



Con questa tecnica, il compilatore suddivide ciascun programma/processo in blocchi di dimensione fissa, detti **pagine**; anche la memoria fisica (RAM) viene suddivisa in blocchi della stessa dimensione, detti **frame**. Ne consegue che ogni frame è in grado di ospitare esattamente una sola pagina. Tipicamente la dimensione più utilizzata oggi giorno è di 4 KB.

Quando un processo deve essere eseguito, le sue pagine vengono caricate in frame qualsiasi della memoria, senza la necessità che questi siano contigui tra di loro; questo risolve il problema della frammentazione esterna, in quanto non si vengono a creare spazi vuoti.

Ovviamente è opportuno tenere traccia, per ogni processo, della posizione di ogni sua pagina all'interno della memoria; a questo scopo è adoperata una **tabella della pagine**, che contiene, per ogni pagina di un processo, l'indirizzo fisico del frame nel quale è stata caricata. Anche con questa tecnica la rilocazione è dinamica ed ogni indirizzo logico viene tradotto in un numero di pagina ed un offset all'interno della pagina; queste



due informazioni vengono utilizzate per accedere correttamente all'indirizzo fisico dove è presente la pagina del processo.

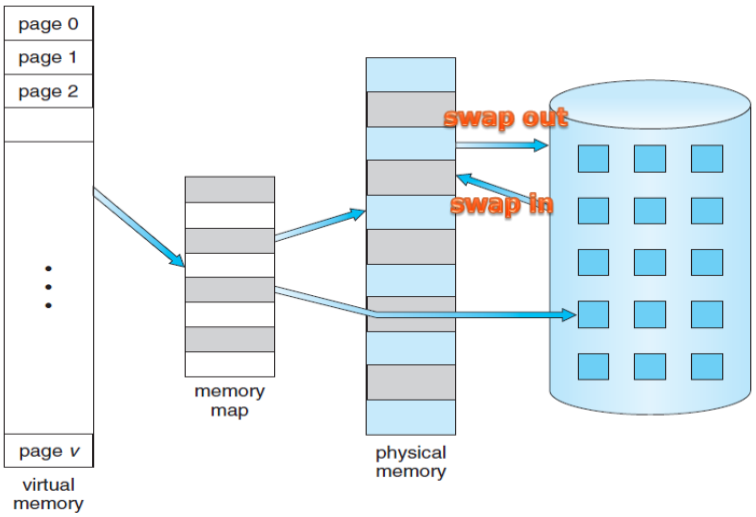
La paginazione è un'ottima tecnica, ma non è priva di difetti: dal momento che ogni processo è suddiviso in pagine della stessa dimensione, l'ultima pagina potrebbe non essere completamente riempita, cosa che porta, seppure in maniera lieve, a frammentazione interna; lo spazio rimanente, infatti, seppur piccolo, non è sfruttabile da altri processi. Inoltre, l'operazione di traduzione da indirizzo logico ad indirizzo fisico aggiunge un costo in termini di tempo, anche se tipicamente è mitigato da una particolare cache molto veloce, detta **TLB** (Translation Lookaside Buffer), che mantiene temporaneamente salvate le associazioni tra indirizzi logici e fisici più utilizzate.

Memoria virtuale

La memoria virtuale è la tecnica prevalentemente utilizzata al giorno d'oggi per la gestione della memoria centrale, che consente al Sistema Operativo di far credere ad ogni processo di avere a disposizione una grande quantità di memoria contigua, anche se in realtà la memoria fisica a disposizione è decisamente più limitata.

È uno schema che cerca di aumentare il grado di multiprogrammazione (avere più processi attivi in memoria) e di consentire l'esecuzione di processi che richiedono più memoria di quella effettivamente disponibile.

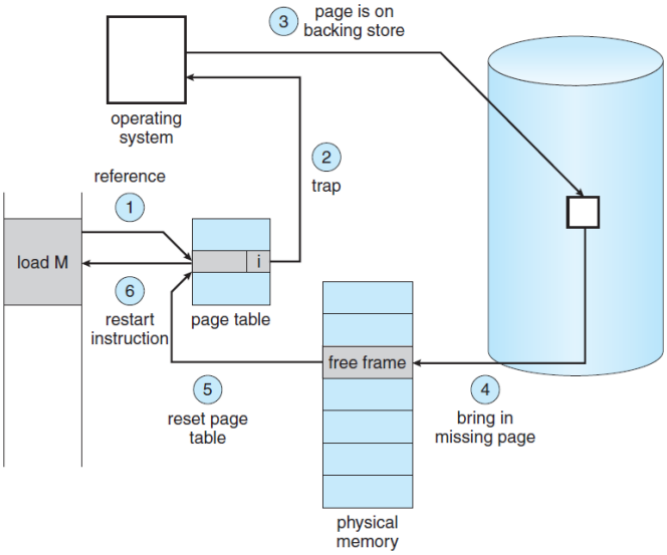
Per raggiungere questi obiettivi si avvale del supporto da parte di un dispositivo di memorizzazione di massa, tipicamente il **disco**: combina la paginazione con un meccanismo chiamato **swapping**. L'idea è quella di avere in memoria RAM solamente quelle porzioni di programma (pagine) che sono effettivamente in uso, mentre quelle restanti vengono lasciate su disco (**backing store**) fino a quando dovranno essere utilizzate.



Questo meccanismo può essere utilizzato anche con altre tecniche precedenti alla paginazione (ad esempio la segmentazione); tuttavia, oltre al problema della frammentazione esterna su RAM introducono frammentazione esterna anche su disco, di gran lunga più grave se consideriamo il fatto che gli accessi e le operazioni su disco sono molto più lente rispetto a quelle su RAM e che, quindi, meccanismi di garbage collection non sono per nulla applicabili.

Anche con questa tecnica la rilocazione è dinamica e avviene nello stesso modo con cui avviene con la paginazione semplice; tuttavia, ora nella tabella delle pagine, per ogni pagina bisogna indicare se queste è effettivamente presente in uno dei frame fisici in memoria, oppure no. Per fare questo, di solito, è utilizzato un **bit di validità**; se la pagina non è presente in RAM, viene generato un "errore" di **page fault**.

Vediamo uno schema che illustra come avviene un page fault:



Come sappiamo, quando viene generato dalla CPU un indirizzo logico, deve essere tradotto in quello fisico utilizzando la tabella delle pagine (1); se la pagina non è presente in memoria RAM (perché è su disco), il bit di validità della pagina sarà settato su "invalida" (i nello schema).

Quindi, l'**MMU** (Memory Management Unit) genera un trap (2), un meccanismo simile ad un interrupt con il quale viene sospesa l'esecuzione del processo e richiesto il caricamento da disco della pagina (3 e 4), che verrà caricata in uno dei frame liberi.

Infine, il bit di validità della pagina viene resettato e viene ripresa l'esecuzione del processo dal punto in cui era stata interrotta.

Prima abbiamo detto che la tecnica della memoria virtuale si basa sul meccanismo di swapping delle pagine; in particolare, l'operazione con cui una pagina caricata su un frame della RAM viene passata su disco è detta **swap out**, l'operazione inversa è detta **swap in**.

Quando una pagina deve essere caricata in memoria RAM, deve essere inserita all'interno di un frame libero; tuttavia, può capitare che tutti i frame siano occupati da una pagina. In questo caso il S.O. deve capire quale frame liberare per fare spazio alla pagina che deve essere caricata in memoria.

Ricordiamo che l'accesso alle memorie di massa (e quindi anche al disco) è molto lento, quindi possiamo provare a pensare ad un meccanismo che ci risparmi l'operazione di salvataggio su disco di una pagina presente in RAM. Possiamo utilizzare un **dirty bit** (o **bit di modifica**), che ci vada ad indicare (per esempio con valore 1) se almeno uno dei byte della pagina è stato modificato (ad esempio è stato cambiato il valore di una variabile); se la pagina viene selezionata per liberare un frame, viene prima controllato il dirty bit: se questo è a 0, allora significa che tutti i byte della pagina sono rimasti invariati, ed è possibile evitare di risalvare la pagina su disco (perché la versione su disco è uguale a quella in RAM), se, invece, è a 1, allora la versione della pagina presente sul disco deve essere aggiornata con quella nuova. Solo dopo possiamo effettuare la sostituzione della pagina.

Strategie di swapping

Se durante la fase di swap in, tutti i frame dovessero essere occupati, occorre selezionare una pagina da riportare su disco (swap out) per fare posto a quella da caricare.

L'operazione di swapping deve essere eseguita tramite algoritmi che consentano di effettuare la scelta di tale pagina nella maniera più intelligente e sofisticata possibile; infatti, se liberiamo un frame e subito dopo la pagina in esso contenuta viene richiesta in RAM, deve essere caricata nuovamente, cosa che porta a dei rallentamenti.

L'ideale sarebbe selezionare la pagina che non verrà richiesta per il più lungo lasso di tempo.

Swapping FIFO

È una strategia molto semplice, che prevede di sostituire la pagina che è presente da più tempo in memoria; tuttavia, non è detto che questa sia necessariamente poco utilizzata.

Swapping LRU

La strategia **LRU** (Least Recently Used) migliora leggermente la logica di quella FIFO: si va a sostituire la pagina che di recente è stata utilizzata di meno. L'idea alla base è che se una pagina è stata utilizzata poco di recente, è facile che anche nell'immediato futuro non venga utilizzata.

Chiaramente anche qui si sta solo scommettendo, non si ha certezza che ciò sia vero.

Ma quanto "di recente" si intende? Chiaramente non possiamo tenere in considerazione "troppo indietro" nel tempo, perché i costi computazionali diverrebbero presto insensati.

Possiamo:

- associare un contatore ad ogni pagina, che venga incrementato ogni volta che tale pagina viene utilizzata e resettato a 0 quando viene ricaricata dal disco, e selezionare per la sostituzione quella pagina con il contatore più basso;
- gestire i riferimenti alle pagine tramite uno stack, dove a livello più alto si trovano le pagina maggiormente utilizzate, e selezionare la pagina che si trova alla base (in fondo) allo stack.